UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2023 Algoritmos y Estructuras de Datos
Diccionarios
Semana 5

Wilder Nina Choquehuayta

wnina@utec.edu.pe

7 de septiembre de 2026

¿Qué aprenderemos hoy?

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Diccionario

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Diccionario
- ¿Qué tan rápido podemos hacerlo con lo que ya sabemos?

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Diccionario
- ¿Qué tan rápido podemos hacerlo con lo que ya sabemos?
- Tablas de direccionamiento directo

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Diccionario
- ¿Qué tan rápido podemos hacerlo con lo que ya sabemos?
- Tablas de direccionamiento directo
- Funciones hash y el problema de las colisiones

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Diccionario
- ¿Qué tan rápido podemos hacerlo con lo que ya sabemos?
- Tablas de direccionamiento directo
- Funciones hash y el problema de las colisiones
- Encadenamiento y direccionamiento abierto

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Diccionario
- ¿Qué tan rápido podemos hacerlo con lo que ya sabemos?
- Tablas de direccionamiento directo
- Funciones hash y el problema de las colisiones
- Encadenamiento y direccionamiento abierto
- Factor de carga y complejidad esperada

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Diccionario
- ¿Qué tan rápido podemos hacerlo con lo que ya sabemos?
- Tablas de direccionamiento directo
- Funciones hash y el problema de las colisiones
- Encadenamiento y direccionamiento abierto
- Factor de carga y complejidad esperada
- Diseñando una buena función hash

El TDA Diccionario

El TDA Diccionario

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un **diccionario** (*map*) guarda pares **(clave, valor)**, donde cada clave aparece a lo más una vez. Soporta:

El TDA Diccionario

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un **diccionario** (*map*) guarda pares **(clave, valor)**, donde cada clave aparece a lo más una vez. Soporta:

- `insertar(clave, valor)`

El TDA Diccionario

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un **diccionario** (*map*) guarda pares **(clave, valor)**, donde cada clave aparece a lo más una vez. Soporta:

- `insertar(clave, valor)`
- `buscar(clave)`: devuelve el valor asociado (o indica que no existe).

El TDA Diccionario

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un **diccionario** (*map*) guarda pares **(clave, valor)**, donde cada clave aparece a lo más una vez. Soporta:

- `insertar(clave, valor)`
- `buscar(clave)`: devuelve el valor asociado (o indica que no existe).
- `eliminar(clave)`

El TDA Diccionario

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un **diccionario** (*map*) guarda pares **(clave, valor)**, donde cada clave aparece a lo más una vez. Soporta:

- `insertar(clave, valor)`
- `buscar(clave)`: devuelve el valor asociado (o indica que no existe).
- `eliminar(clave)`

Ejemplos de uso

Contar cuántas veces aparece cada palabra en un texto; guardar el nombre de usuario asociado a cada ID; una caché de resultados ya calculados.

¿Qué tan rápido podemos hacerlo ya?

Repaso: nuestras herramientas actuales

¡Piénsalo!

Con arreglos y listas enlazadas (lo que ya conocemos), ¿qué complejidad logramos para `insertar`/`buscar`/ `eliminar` por clave?

Repaso: nuestras herramientas actuales



	Buscar	Insertar	Eliminar

Repaso: nuestras herramientas actuales

	Buscar	Insertar	Eliminar
Arreglo/lista, sin ordenar	$O(n)$	$O(1)$	$O(n)$

Repaso: nuestras herramientas actuales

	Buscar	**Insertar**	**Eliminar**
Arreglo/lista, sin ordenar	$O(n)$	$O(1)$	$O(n)$
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$

Repaso: nuestras herramientas actuales

	Buscar	**Insertar**	**Eliminar**
Arreglo/lista, sin ordenar	$O(n)$	$O(1)$	$O(n)$
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$

Ninguna opción logra las tres en $O(1)$ o incluso $O(\lg n)$ **simultáneamente** — ordenar ayuda a buscar, pero encarece insertar/eliminar (hay que desplazar), exactamente el mismo dilema que ya vimos con arreglos vs. listas enlazadas.

Repaso: nuestras herramientas actuales

	Buscar	Insertar	Eliminar
Arreglo/lista, sin ordenar	$O(n)$	$O(1)$	$O(n)$
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$

Ninguna opción logra las tres en $O(1)$ o incluso $O(\lg n)$ **simultáneamente** — ordenar ayuda a buscar, pero encarece insertar/eliminar (hay que desplazar), exactamente el mismo dilema que ya vimos con arreglos vs. listas enlazadas.

La meta de hoy

Lograr las tres operaciones en $O(1)$ **esperado** — no en el peor caso, pero sí en la inmensa mayoría de los casos reales.

Tablas de direccionamiento directo

El caso ideal: acceso directo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Si las claves fueran enteros pequeños

Supongamos que las claves son enteros en $\{0, 1, \ldots, m-1\}$. Entonces basta un arreglo $T[0..m-1]$: guardar el valor de la clave k directamente en $T[k]$.

El caso ideal: acceso directo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Si las claves fueran enteros pequeños

Supongamos que las claves son enteros en $\{0, 1, \ldots, m-1\}$. Entonces basta un arreglo $T[0..m-1]$: guardar el valor de la clave k directamente en $T[k]$.

Complejidad

`insertar`, `buscar` y `eliminar` son $O(1)$ **exactos** — indexar un arreglo.

El caso ideal: acceso directo

Si las claves fueran enteros pequeños

Supongamos que las claves son enteros en $\{0, 1, \ldots, m-1\}$. Entonces basta un arreglo $T[0..m-1]$: guardar el valor de la clave k directamente en $T[k]$.

Complejidad

`insertar`, `buscar` y `eliminar` son $O(1)$ **exactos** — indexar un arreglo.

Esto se llama una **tabla de direccionamiento directo**. ¿Cuál es el problema?

El problema: el universo de claves es enorme

¡Piénsalo!

Si las claves son, por ejemplo, códigos de estudiante de 9 dígitos (10^9 valores posibles), pero en la práctica solo tienes $n = 200$ estudiantes, ¿qué pasa si reservas un arreglo del tamaño de **todo** el universo de claves?

El problema: el universo de claves es enorme

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si las claves son, por ejemplo, códigos de estudiante de 9 dígitos (10^9 valores posibles), pero en la práctica solo tienes $n = 200$ estudiantes, ¿qué pasa si reservas un arreglo del tamaño de **todo** el universo de claves?

Respuesta

Se desperdicia una cantidad enorme de memoria (un arreglo de 10^9 posiciones para guardar 200 valores) — y ni hablar si las claves son **strings** (nombres, palabras), donde ni siquiera hay un orden natural de indexación directa.

Funciones hash

La idea: comprimir el universo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Función hash

Una **función hash** h mapea el universo (grande) de claves posibles U a un rango (chico) de índices de tabla:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}.$$

Guardamos la clave k en la posición $T[h(k)]$, con $m \ll |U|$.

La idea: comprimir el universo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Función hash

Una **función hash** h mapea el universo (grande) de claves posibles U a un rango (chico) de índices de tabla:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}.$$

Guardamos la clave k en la posición $T[h(k)]$, con $m \ll |U|$.

Ahora la tabla mide m (proporcional a cuántas claves **realmente** usamos), no $|U|$.

El problema de las colisiones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si $|U| > m$ (el universo tiene más elementos que la tabla), ¿es posible que h sea **inyectiva** (que nunca dos claves distintas caigan en la misma posición)?

El problema de las colisiones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si $|U| > m$ (el universo tiene más elementos que la tabla), ¿es posible que h sea **inyectiva** (que nunca dos claves distintas caigan en la misma posición)?

Respuesta: no, nunca

Por el **principio del palomar**: si metes más de m claves posibles en m casillas, al menos dos deben compartir casilla. Una **colisión** ocurre cuando $h(k_1) = h(k_2)$ con $k_1 \neq k_2$.

El problema de las colisiones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

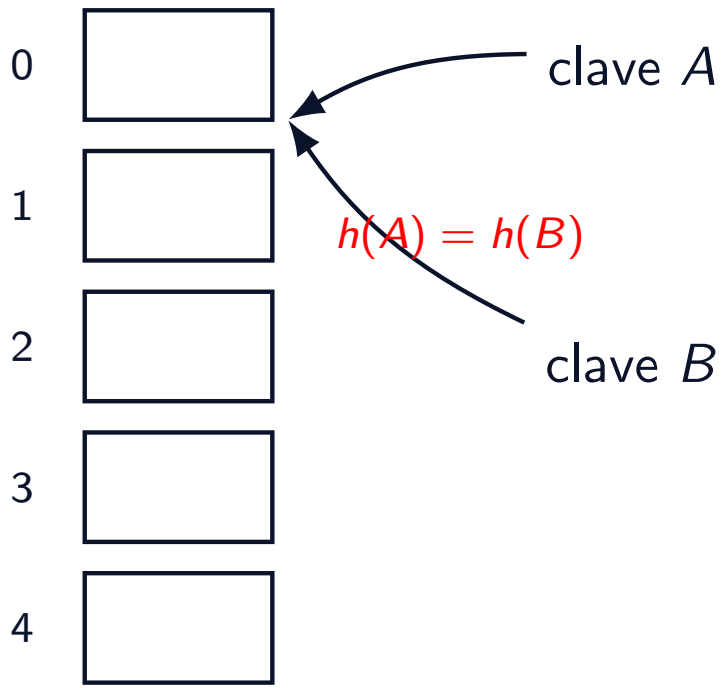
Si $|U| > m$ (el universo tiene más elementos que la tabla), ¿es posible que h sea **inyectiva** (que nunca dos claves distintas caigan en la misma posición)?

Respuesta: no, nunca

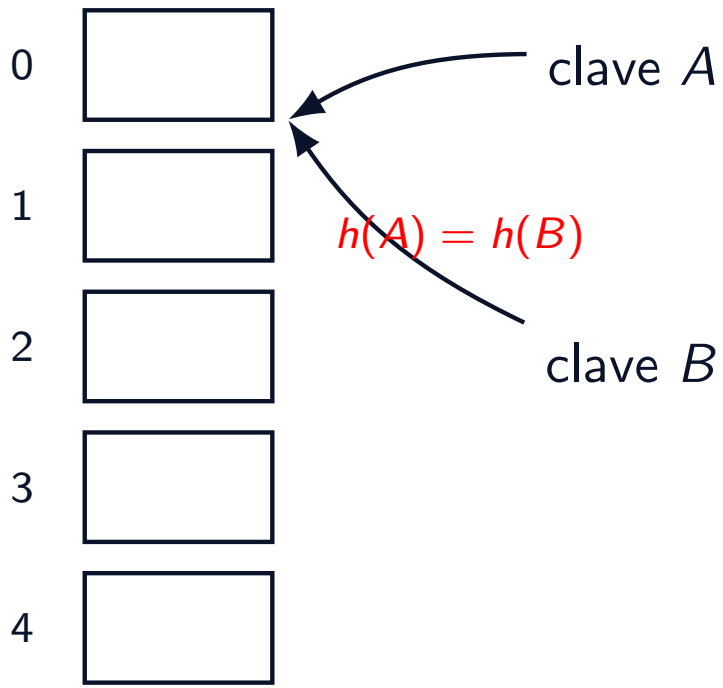
Por el **principio del palomar**: si metes más de m claves posibles en m casillas, al menos dos deben compartir casilla. Una **colisión** ocurre cuando $h(k_1) = h(k_2)$ con $k_1 \neq k_2$.

Las colisiones son **inevitables** en general — lo que podemos controlar es qué tan **seguido** ocurren, y qué hacer cuando ocurren.

Visualizando una colisión



Visualizando una colisión



Dos claves distintas, A y B, se hashean a la **misma** posición — necesitamos una estrategia para que ambas convivan en la tabla.

Resolviendo colisiones

Estrategia 1: encadenamiento separado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Chaining

Cada posición $T[i]$ de la tabla guarda, no un solo valor, sino una **lista enlazada** con todas las claves que hashean a i.

Estrategia 1: encadenamiento separado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

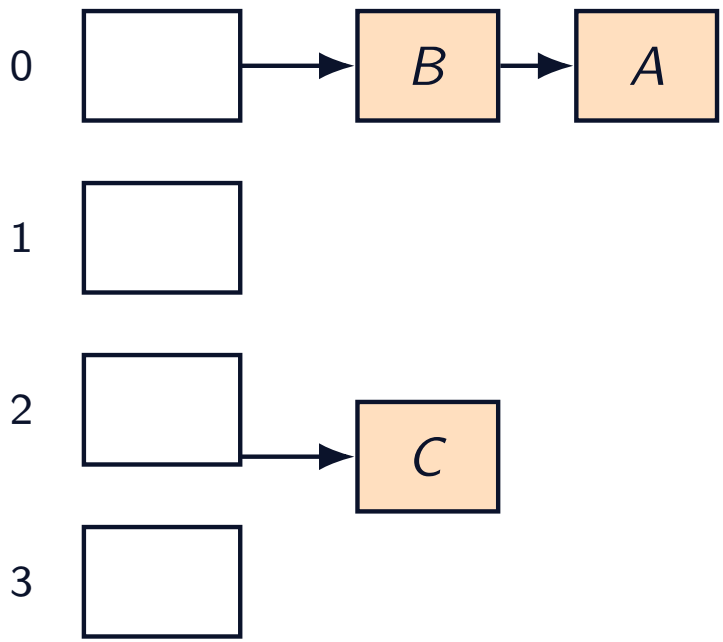
Chaining

Cada posición $T[i]$ de la tabla guarda, no un solo valor, sino una **lista enlazada** con todas las claves que hashean a i.

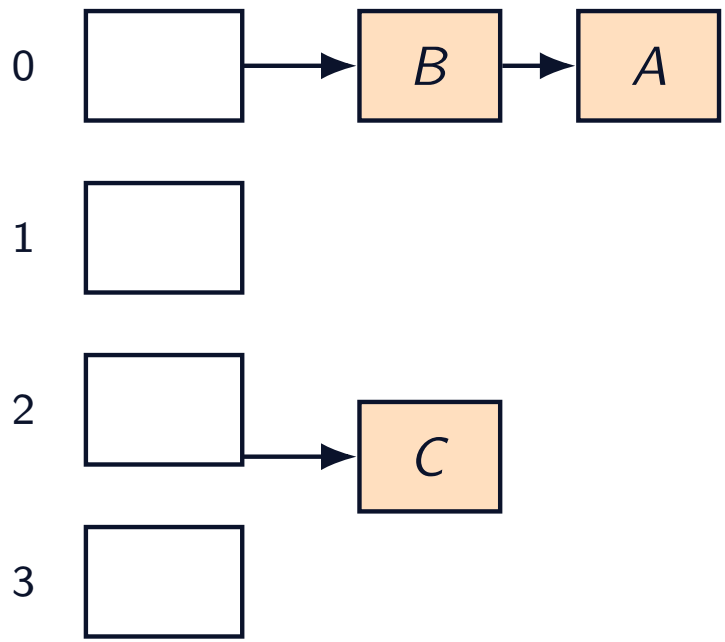
¿Por qué nos conviene justo ahora?

Ya sabemos implementar listas enlazadas (clase pasada) — `insertar` en un bucket es un `push_front`, `buscar` recorre la lista de ese bucket. La próxima clase la implementamos así.

Visualizando el encadenamiento



Visualizando el encadenamiento



Las claves A y B colisionaron en la posición 0: ambas viven en la lista enlazada de ese *bucket*. La posición 3 solo tiene a C.

Estrategia 2: direccionamiento abierto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Open addressing

En vez de listas, **todo** vive dentro del arreglo T: si la posición $h(k)$ ya está ocupada, se prueba otra posición siguiendo una secuencia fija (*probing*), hasta encontrar una libre.

Estrategia 2: direccionamiento abierto

Open addressing

En vez de listas, **todo** vive dentro del arreglo T: si la posición $h(k)$ ya está ocupada, se prueba otra posición siguiendo una secuencia fija (*probing*), hasta encontrar una libre.

Variantes (solo para que las reconozcas)

Sondeo lineal: probar $h(k)+1, h(k)+2, \ldots$ **Sondeo cuadrático**: probar $h(k)+1^2, h(k)+2^2, \ldots$ **Hashing doble**: usar una segunda función hash para el tamaño del salto.

Estrategia 2: direccionamiento abierto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Open addressing

En vez de listas, **todo** vive dentro del arreglo T: si la posición $h(k)$ ya está ocupada, se prueba otra posición siguiendo una secuencia fija (*probing*), hasta encontrar una libre.

Variantes (solo para que las reconozcas)

Sondeo lineal: probar $h(k)+1, h(k)+2, \ldots$ **Sondeo cuadrático**: probar $h(k)+1^2, h(k)+2^2, \ldots$ **Hashing doble**: usar una segunda función hash para el tamaño del salto.

No lo implementaremos en este curso (nos quedamos con encadenamiento, más simple de razonar) — pero `std::unordered_map` y tablas hash de otros lenguajes a veces usan variantes de direccionamiento abierto.

Factor de carga y complejidad

Factor de carga

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Con encadenamiento, el **factor de carga** es

$$\alpha = \frac{n}{m},$$

donde n es el número de claves guardadas y m el número de posiciones (*buckets*) de la tabla — es decir, el **promedio** de claves por bucket.

Factor de carga

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Con encadenamiento, el **factor de carga** es

$$\alpha = \frac{n}{m},$$

donde n es el número de claves guardadas y m el número de posiciones (*buckets*) de la tabla — es decir, el **promedio** de claves por bucket.

¡Piénsalo!

Si buscas una clave y su bucket tiene, en promedio, α elementos, ¿qué tan larga es la lista que hay que recorrer en el peor caso **de ese bucket**?

Tiempo esperado: $\Theta(1+\alpha)$

Bajo hashing uniforme simple

Suponiendo que h distribuye las claves de forma pareja entre los m buckets (cada clave, independientemente, es igual de probable de caer en cualquier bucket): el número **esperado** de claves en un bucket dado es $\alpha = n/m$.

Tiempo esperado: $\Theta(1+\alpha)$

Bajo hashing uniforme simple

Suponiendo que h distribuye las claves de forma pareja entre los m buckets (cada clave, independientemente, es igual de probable de caer en cualquier bucket): el número **esperado** de claves en un bucket dado es $\alpha = n/m$.

Conclusión

Buscar una clave cuesta $\Theta(1)$ (calcular $h(k)$) más recorrer, en promedio, $\Theta(\alpha)$ elementos del bucket:

$$\Theta(1+\alpha).$$

Tiempo esperado: $\Theta(1+\alpha)$

UTEC
UNIVERSIDAD DE INGENIERÍA

Bajo hashing uniforme simple

Suponiendo que h distribuye las claves de forma pareja entre los m buckets (cada clave, independientemente, es igual de probable de caer en cualquier bucket): el número **esperado** de claves en un bucket dado es $\alpha = n/m$.

Conclusión

Buscar una clave cuesta $\Theta(1)$ (calcular $h(k)$) más recorrer, en promedio, $\Theta(\alpha)$ elementos del bucket:

$$\Theta(1+\alpha).$$

La clave para lograr $O(1)$

Si mantenemos m proporcional a n (por ejemplo, **duplicando** m cuando α se vuelve grande — exactamente como el crecimiento amortizado de `vector` y de la cola circular), $\alpha = O(1)$ **siempre**, y entonces $\Theta(1+\alpha) = \Theta(1)$.

¿Y si la función hash es mala?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si $h(k) = 0$ para **toda** clave k (una función hash pésima, pero válida), ¿qué le pasa a la complejidad, sin importar qué tan grande sea m?

¿Y si la función hash es mala?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Si $h(k) = 0$ para **toda** clave k (una función hash pésima, pero válida), ¿qué le pasa a la complejidad, sin importar qué tan grande sea m?

Respuesta

Todas las claves caen en el **mismo** bucket: se degrada a una sola lista enlazada de n elementos — $\Theta(n)$ por operación, el peor caso posible. El $\Theta(1 + \alpha)$ esperado **depende** de que h reparta las claves de forma pareja.

Diseñando una función hash

Método de la división

$h(k) = k$ mód m

Para claves enteras: el resto de dividir k entre m. Simple y rápido.

Método de la división

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$h(k) = k$ mód m

Para claves enteras: el resto de dividir k entre m. Simple y rápido.

¡Piénsalo!

Si $m = 2^{10}$ (una potencia de 2) y las claves son, por ejemplo, direcciones de memoria (casi siempre pares, o múltiplos de 4 u 8), ¿qué le pasa a la distribución de $h(k)$?

Método de la división

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$h(k) = k$ mód m

Para claves enteras: el resto de dividir k entre m. Simple y rápido.

¡Piénsalo!

Si $m = 2^{10}$ (una potencia de 2) y las claves son, por ejemplo, direcciones de memoria (casi siempre pares, o múltiplos de 4 u 8), ¿qué le pasa a la distribución de $h(k)$?

Respuesta: se concentra mal

k mód 2^{10} solo depende de los últimos 10 bits de k — si esos bits tienen un patrón (por ejemplo, siempre pares), muchas claves distintas comparten los mismos residuos. **Recomendación**: elegir m **primo**, alejado de potencias de 2.

Ejemplo trabajado: construyendo la tabla

$m = 7$ (primo), claves $\{23, 45, 12, 37, 7, 20\}$

Calculando $h(k) = k$ mód 7 para cada una:

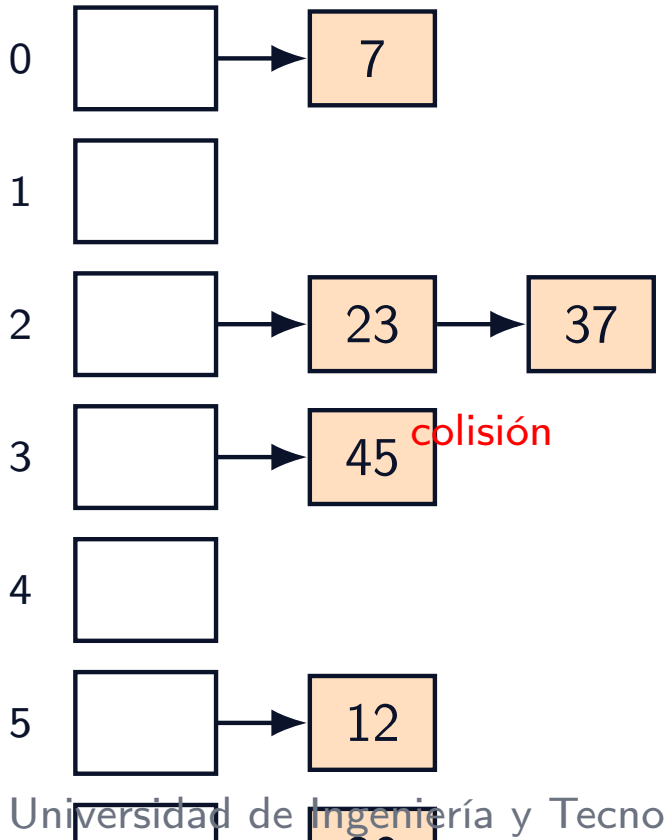
$$h(23)=2,\ \ h(45)=3,\ \ h(12)=5,\ \ h(37)=2,\ \ h(7)=0,\ \ h(20)=6.$$

Ejemplo trabajado: construyendo la tabla

$m = 7$ (primo), claves $\{23, 45, 12, 37, 7, 20\}$

Calculando $h(k) = k$ mód 7 para cada una:

$$h(23)=2,\ h(45)=3,\ h(12)=5,\ h(37)=2,\ h(7)=0,\ h(20)=6.$$

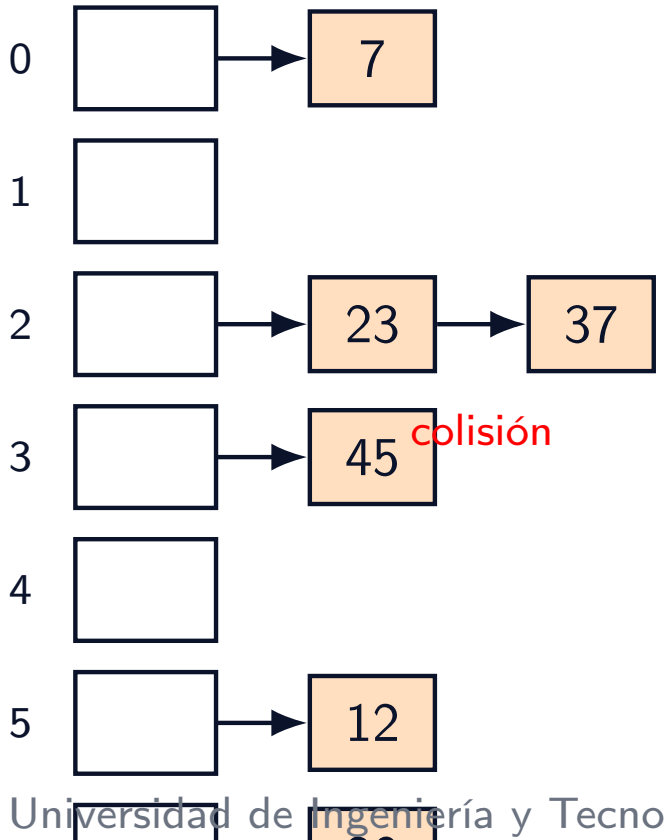


Ejemplo trabajado: construyendo la tabla

$m = 7$ (primo), claves $\{23, 45, 12, 37, 7, 20\}$

Calculando $h(k) = k$ mód 7 para cada una:

$$h(23)=2,\ h(45)=3,\ h(12)=5,\ h(37)=2,\ h(7)=0,\ h(20)=6.$$



Método de la multiplicación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$

Con A una constante irracional bien elegida (por ejemplo, $A \approx (\sqrt{5} - 1)/2$, la razón áurea). kA mód 1 es la parte fraccionaria de kA.

Método de la multiplicación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$

Con A una constante irracional bien elegida (por ejemplo, $A \approx (\sqrt{5} - 1)/2$, la razón áurea). kA mód 1 es la parte fraccionaria de kA.

Ventaja: el valor de m **no** es tan delicado como en el método de la división (funciona razonablemente para casi cualquier m, incluyendo potencias de 2, cómodas para la aritmética de computadora).

Hashing de *strings*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Nuestras funciones hash asumen una clave **numérica**. ¿Cómo hasheamos un *string* (un nombre, una palabra)?

Hashing de *strings*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Nuestras funciones hash asumen una clave **numérica**. ¿Cómo hasheamos un *string* (un nombre, una palabra)?

Idea: tratarlo como un número en base B

Para $s = s_0 s_1 \dots s_{L-1}$ (códigos ASCII/Unicode), definir

$$\text{val}(s) = \sum_{i=0}^{L-1} s_i \cdot B^i \quad (\text{mód } M),$$

con B una base (por ejemplo 131 o 257, mayor que el alfabeto) y M grande. Luego aplicamos $h(\text{val}(s)) = \text{val}(s)$ mód m como antes.

Propiedades de una buena función hash

Propiedades de una buena función hash

- **Determinística**: la misma clave siempre produce el mismo índice.

Propiedades de una buena función hash

- **Determinística**: la misma clave siempre produce el mismo índice.
- **Rápida de calcular**: $O(1)$ respecto al tamaño de la clave (o lineal en su longitud, para strings).

Propiedades de una buena función hash

- **Determinística**: la misma clave siempre produce el mismo índice.
- **Rápida de calcular**: $O(1)$ respecto al tamaño de la clave (o lineal en su longitud, para strings).
- **Distribución uniforme**: claves distintas deben repartirse parejo entre los m buckets, sin importar patrones que pueda tener el conjunto real de claves usadas.

Aplicaciones

Aplicaciones

- `dict` de Python, `unordered_map` de C++, `HashMap` de Java: todos son tablas hash.

Aplicaciones

- `dict` de Python, `unordered_map` de C++, `HashMap` de Java: todos son tablas hash.
- **Conteo de frecuencias**: ¿cuántas veces aparece cada palabra en un texto?

Aplicaciones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- `dict` de Python, `unordered_map` de C++, `HashMap` de Java: todos son tablas hash.
- **Conteo de frecuencias**: ¿cuántas veces aparece cada palabra en un texto?
- **Detección de duplicados**: ¿hay dos elementos iguales en un arreglo?

Aplicaciones

- `dict` de Python, `unordered_map` de C++, `HashMap` de Java: todos son tablas hash.
- **Conteo de frecuencias**: ¿cuántas veces aparece cada palabra en un texto?
- **Detección de duplicados**: ¿hay dos elementos iguales en un arreglo?
- **Caché**: guardar resultados ya calculados, indexados por sus argumentos de entrada.

Aplicaciones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- `dict` de Python, `unordered_map` de C++, `HashMap` de Java: todos son tablas hash.
- **Conteo de frecuencias**: ¿cuántas veces aparece cada palabra en un texto?
- **Detección de duplicados**: ¿hay dos elementos iguales en un arreglo?
- **Caché**: guardar resultados ya calculados, indexados por sus argumentos de entrada.
- Índices de bases de datos, tablas de símbolos de compiladores, y muchísimo más.

Resumen de la sesión



Resumen de la sesión

- Un **diccionario** necesita `insertar`/ `buscar`/`eliminar` rápido por clave — ninguna estructura vista hasta ahora logra las tres a la vez.

Resumen de la sesión

- Un **diccionario** necesita `insertar`/ `buscar`/`eliminar` rápido por clave — ninguna estructura vista hasta ahora logra las tres a la vez.
- Una **función hash** $h : U \to \{0, \ldots, m-1\}$ comprime un universo grande de claves a una tabla chica.

Resumen de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un **diccionario** necesita `insertar`/ `buscar`/`eliminar` rápido por clave — ninguna estructura vista hasta ahora logra las tres a la vez.
- Una **función hash** $h : U \to \{0, \ldots, m-1\}$ comprime un universo grande de claves a una tabla chica.
- Las **colisiones** son inevitables (principio del palomar); se resuelven con **encadenamiento** (listas por bucket) o **direccionamiento abierto** (*probing*).

Resumen de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un **diccionario** necesita `insertar`/ `buscar`/`eliminar` rápido por clave — ninguna estructura vista hasta ahora logra las tres a la vez.
- Una **función hash** $h : U \to \{0, \ldots, m-1\}$ comprime un universo grande de claves a una tabla chica.
- Las **colisiones** son inevitables (principio del palomar); se resuelven con **encadenamiento** (listas por bucket) o **direccionamiento abierto** (*probing*).
- Con hashing uniforme simple y factor de carga $\alpha = n/m$ acotado (creciendo la tabla cuando hace falta), toda operación es $\Theta(1 + \alpha) = \Theta(1)$ esperado.

Resumen de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un **diccionario** necesita `insertar`/ `buscar`/`eliminar` rápido por clave — ninguna estructura vista hasta ahora logra las tres a la vez.
- Una **función hash** $h : U \to \{0, \ldots, m-1\}$ comprime un universo grande de claves a una tabla chica.
- Las **colisiones** son inevitables (principio del palomar); se resuelven con **encadenamiento** (listas por bucket) o **direccionamiento abierto** (*probing*).
- Con hashing uniforme simple y factor de carga $\alpha = n/m$ acotado (creciendo la tabla cuando hace falta), toda operación es $\Theta(1 + \alpha) = \Theta(1)$ esperado.
- **Próxima clase**: implementamos una tabla hash con encadenamiento, usando las listas enlazadas que ya construimos.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias